

C++ Template 延伸自主練習作業 學習報告

GitHub repository:

https://github.com/cyt-0/cpp_StudentSystem

學號:S11459013

姓名:陳禹彤

指導教授:林朝興 教授

壹、 自主學習主題與目標

本次自主學習的主題為「C++ STL 與基礎資料結構入門」。目標是透過延伸 Chapter 16 Templates 的觀念，進階學習 C++標準模板函式庫(STL)的使用方式。

主要的學習目標包含：

- 理解 Template 與 STL 之間的底層關係。
- 熟悉並實作常見的 STL 容器(vector 動態陣列)。
- 學習使用<algorithm>中的 sort()演算法搭配 Lambda 運算式。
- 透過實作「學生成績管理系統」，將上述抽象觀念轉化為具體的整合應用。

貳、 Template 與 STL 的關係

在 C++中，Template 是一種泛型程式設計(Generic Programming)的技術。它允許開發者撰寫與「資料型別」無關的程式碼。例如，我們可以寫一個 getMax 函式，透過 Template 讓它同時支援比較 int、float 甚至是自定義的結構。

STL 則是建立在 Template 觀念之上的一套強大工具包。STL 內部充滿了各種資料結構與演算法。因為使用了 Class Template 的技術，所以像是 vector 這樣的容器，才能透過 vector<int> 或 vector<Student>這樣的語法，裝載各式各樣不同型別的資料，而不需要為每種資料型別重新撰寫一套陣列管理程式。

參、常見 STL 容器用途整理

- vector(動態陣列):

在記憶體中是連續排列的，支援隨機存取(透過索引值快速抓取資料)。當容量不足時，它會自動在記憶體中尋找更大的空間並搬移資料。適合需要頻繁走訪、排序的資料。

- stack(堆疊):

具有「後進先出(LIFO, Last In First Out)」的特性。只能從頂端推入或彈出資料。常用於括號配對檢查、網頁上一頁功能，或是深度優先搜尋(DFS)。

- queue(佇列):

具有「先進先出(FIFO, First In First Out)」的特性。像是在便利商店排隊一樣，先加入的元素先被處理。常用於模擬排隊系統或是廣度優先搜尋(BFS)。

- map(映射表):

以鍵值對(Key-Value)的方式儲存資料，底層通常是紅黑樹實作。可以透過特定的 Key(例如字串或學號)快速尋找對應的 Value，適合用來做字典或統計單字出現次數。

肆、第 2 週小專題設計說明

本次專題實作「學生成績管理系統」，主要架構設計如下:

- 資料儲存:使用 struct Student 來封裝學生的學號(id)、姓名(name)與成績(score)。考量到系統需要實作「依成績排序」的功能，因此選擇使用 vector<Student>作為主要的 STL 容器，方便套用標準排序演算法。
- 五大功能模組:

1. 新增學生:實作防呆機制，新增前會先使用迴圈檢查 vector 中是否有重複的學號，若有則拒絕新增並報錯。

2. 列出學生:透過 for 迴圈走訪陣列，將所有物件的屬性格式化輸出。
3. 依成績排序:呼叫 STL 的 sort()進行由高至低排序。
4. 查詢學生:使用線性搜尋比對學號。
5. 統計成績:整合自行設計的 Template 函式 getMax 與 getMin 來抓出極值，並計算平均與及格人數。

伍、 主要程式架構與重要程式片段

本系統的架構採用模組化思維，將資料結構、泛型函式與各項操作功能分開設計，並透過一個無窮迴圈 while(true) 搭配 switch-case 來建立終端機的互動選單。以下針對系統中的五大核心架構進行詳細的程式碼解析與設計說明：

- 系統核心資料結構設計：

為了有效管理學生的各項屬性，系統首先定義了 Student 結構 (Struct)，將學生的學號 (id)、姓名 (name) 與成績 (score) 封裝在一起。

在容器的選擇上，本系統採用了 STL 的 vector<Student>作為主要的資料儲存庫。vector 具備動態調整大小的優勢，能完美應付未知的學生輸入數量，同時支援隨機存取，讓後續的線性搜尋與排序演算法更方便。

```
struct Student
{
    string id;
    string name;
    int score;
};
```

- Template 實作:

使用了 getMax 與 getMin 兩個 Template 函式。這裡運用了直覺且易讀的 if-else 判斷邏輯來進行數值大小的比對。透過 Template 的特性，未來若系統需要擴充，此函式皆能直接套用，無須為不同的資料型別重新定義函式。

```
template <class T>
T getMax(T a,T b)
{
    if(a>b)
    {
        return a;
    }
    else
    {
        return b;
    }
}

template <class T>
T getMin(T a,T b)
{
    if(a<b)
    {
        return a;
    }
    else
    {
        return b;
    }
}
```

- 新增學生與重複學號防呆(Case 1):

在「新增學生」的功能中，為了確保資料庫的唯一性（不出現重複的學號），系統實作了嚴謹的防呆機制。在接收使用者輸入的學號後，程式會先設定一個 is_duplicate 旗標，並利用 for 迴圈走訪當前 vector 內的所有元素。

若發現相同的學號，便會將旗標設為 true 並提早 break 結束迴圈，隨後拒絕該次新增並印出錯誤提示；唯有確認學號不重複時，才會繼續請求姓名與成績，並使用 push_back() 安全地寫入容器尾端。

```
bool is_duplicate=false;
for(int i=0;i<students.size();i++)
{
    if(students[i].id==new_stu.id)
    {
        is_duplicate=true;
        break;
    }
}
if(is_duplicate)
{
    cout<<"ERROR:This ID already exists!"<<endl;
}
else
{
    cout<<"Enter student's name: "<<endl;
    cin>>new_stu.name;
    cout<<"Enter student's score: "<<endl;
    cin>>new_stu.score;

    students.push_back(new_stu);
    cout<<"Add Success!"<<endl;
}

break;
```

- 自訂條件排序：

依成績排序功能是直接引入<algorithm>函式庫中的 sort()。為了讓編譯器知道要根據 Student 結構中的哪一個屬性進行排序，採用了 [](const Student& a,const Student& b)作為自訂的比較規則。透過回傳 a.score > b.score，系統能精準地將物件陣列依照成績由高至低進行重新排列。

```
sort(students.begin(),students.end(),[](const Student& a,const Student& b)
{
    return a.score>b.score;
});
cout<<"Sort Success!"<<endl;
break;
```

陸、 執行畫面或輸出結果

- 主選單畫面

```
---Student Performance Management System---
1. Add Student
2. List All Students
3. Sort By Score
4. Search By ID
5. Show Statistics
0. Exit
Enter Your Choice:
```

- 新增學生成功畫面

```
Enter Your Choice:
1
Enter ID:
11459013
Enter student's name:
yutung
Enter student's score:
85
Add Success!
```

- 重複學號防呆錯誤提示畫面

```
Enter Your Choice:
1
Enter ID:
11459013
ERROR:This ID already exists!
```

- 列出所有學生資料畫面

```
Enter Your Choice:
2
---Student List---
ID: 11459013
Name: yutung
Score: 85

ID: 11459014
Name: mark
Score: 79

ID: 11459015
Name: rita
Score: 88

ID: 11459010
Name: amy
Score: 90
```

- 依成績排序完成，並重新列出之畫面

```
Enter Your Choice:
3
Sort Success!
---Student Performance Management System---
1. Add Student
2. List All Students
3. Sort By Score
4. Search By ID
5. Show Statistics
0. Exit
Enter Your Choice:
2
---Student List---
ID: 11459010
Name: amy
Score: 90

ID: 11459015
Name: rita
Score: 88

ID: 11459013
Name: yutung
Score: 85

ID: 11459014
Name: mark
Score: 79
```

- 以學號查詢特定學生畫面

```
Enter Your Choice:
4
Enter the student's ID you want to find:
11459015
ID: 11459015
Name: rita
Score: 88
```

- 全班成績統計與極值顯示畫面

```
Enter Your Choice:
5
Average Score: 85.5
Highest Score: 90
Lowest Score: 79
Pass: 4
Fail: 0
```

柒、 GitHub repo link 與檔案結構說明

- GitHub Repository 連結：

https://github.com/cyt-0/cpp_StudentSystem

- 檔案結構說明：

1. main.cpp：本專題的核心程式碼，包含所有的邏輯處理、STL 呼叫與終端機互動介面。
2. README.md：專案的說明文件，內部清楚記載了系統的功能簡介，以及如何在支援 C++17 的環境下進行編譯與執行的具體指令。
3. report.pdf：本份學習報告。

捌、 遇到的問題與解決方式

switch-case 內的變數宣告範圍問題：

- 問題描述：在最初撰寫選單架構時，我在 case 1 裡面直接宣告了 Student new_stu;，準備接收使用者輸入。結果在執行編譯指令時，編譯器卻跳出了錯誤，指出變數初始化的範圍有問題。

- 原因分析：經過研究後發現，在 switch 敘述中，所有的 case 其實都屬於同一個區域範圍。如果在某個 case 中宣告了新變數，編譯器會擔心如果程式發生「穿透 (Fall-through)」，這個變數的生命週期會不小心延續並干擾到其他的 case。
- 解決方案：利用大括號 {} 將每一個 case 的執行內容獨立包裝起來，為每個選項建立專屬的區域範圍。

玖、學習心得與未來可延伸方向。

- 學習心得：

經過這兩週的自主學習與小專題實作，我對 C++ 的認知有了顯著的轉變。利用 vector 來處理動態資料增長，徹底解決了以往使用傳統 C 語言陣列時，總是需要提心吊膽預估記憶體大小的困擾；而 <algorithm> 函式庫中的 sort() 搭配 Lambda 運算式的寫法，不僅大幅縮短了程式碼長度，更讓我體會到「將演算法與資料結構解耦」的優雅設計思維。

這次實作的學生成績管理系統雖然功能基礎，但它完整涵蓋了資料結構定義 (struct)、泛型設計 (template)、動態記憶體管理 (vector) 以及防呆邏輯 (重複學號檢查)。透過親手撰寫這套系統，並逐步排除編譯錯誤與邏輯穿透 (Fall-through) 的 Bug，我確信自己為未來大二的資料結構課程打下了非常紮實的底子。

- 未來可延伸方向：

1. 複雜實驗數據處理：

未來如果需要處理大量的科學數據，例如之前做物理電路實驗時收集的歐姆定律或電路定理的數據集，我就能利用這次學到的 vector 快速儲存浮點數資料，並利用 sort 與 Template 函式進行極值分析與排序，將這套系統轉化為泛用的實驗數據分析工具。

2. 導入 map 進行搜尋效能優化：

在本次專題中，查詢學生的功能是使用 vector 搭配迴圈進行線性搜尋。未來若系統擴充至全校等級的資料量，我計畫將資料結構改為

`map<string, Student>`，以學號作為 Key 值。這樣一來，無論資料量多龐大，系統都能極快速度精準定位到特定學生的資料。

3. 資料持久化與檔案 I/O：

目前系統的資料皆儲存於暫存記憶體中，程式結束即消失。未來可進一步學習 `<fstream>` 函式庫，讓系統具備讀寫.csv 或.txt 檔案的能力，實現真正的資料持久化儲存。